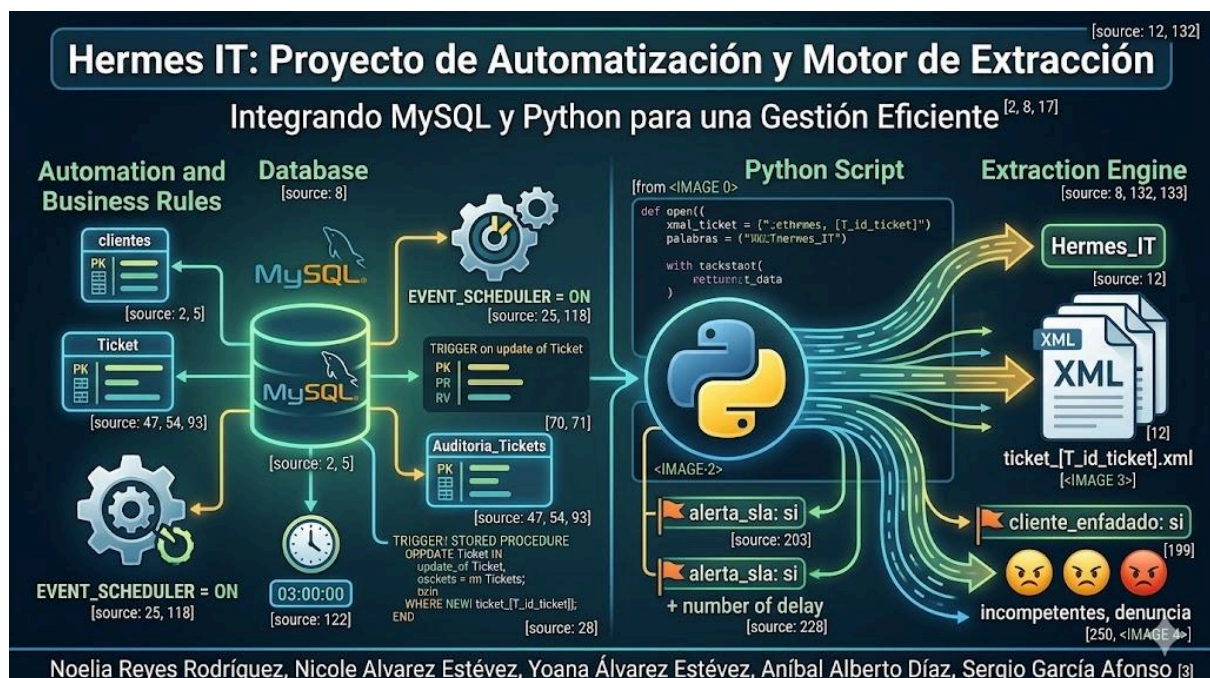


Semana 2: Automatización y motor de extracción



Noelia Reyes Rodríguez
Nicole Álvarez Estévez
Yoana Álvarez Estévez
Aníbal Alberto Díaz
Sergio García Afonso

Índice

Semana 2: Automatización y motor de extracción.....	1
Índice.....	2
Repartición de funciones.....	3
Introducción.....	3
FOL.....	3
Automatización y Reglas de Negocio.....	4
1. Creación del procedimiento.....	4
2. Creación del trigger.....	5
3. Creación de tabla.....	6
4. Creación de evento.....	7
Batería de Pruebas.....	8
Insertar Ticket.....	8
Insertar Ticket con ID existente.....	8
Intentar cerrar un ticket sin historial de mensaje.....	9
Registro de cambios de estados del ticket.....	9
Evento.....	10
Motor de Extracción y Transformación a XML.....	10
1. Extractor.....	10
2. Funciones de enfado.....	15
3. Modificar la descripción del ticket.....	16

Repartición de funciones

Base de Datos: Nicole, Aníbal y Noelia

Script en Python: Yoana y Sergio

Subir a GitHub: Noelia

Documentación e informe: Noelia, Nicole y Yoana

Comprobación, puesta en común y mejoras: Todos

Introducción

En el proyecto Hermes IT hemos mejorado el sistema de tickets para que funcione de forma más automática y segura. Ahora la base de datos no sólo guarda información, sino que también ayuda a gestionar mejor los datos y evita errores importantes, como cerrar tickets sin información o tener que hacer varios pasos manuales para crear clientes y tickets.

También se ha añadido un sistema que guarda cambios importantes y que, con el tiempo, archiva automáticamente los tickets antiguos para mantener la base de datos más ordenada y rápida.

Por otro lado, hemos creado un programa en Python que saca esos tickets archivados y los convierte en archivos XML. Además, este programa añade información extra, como si un ticket se ha tardado mucho en resolver o si el cliente ha mostrado enfado en sus mensajes.

En conclusión, el sistema ahora es más automático, está mejor organizado y permite guardar y usar la información de una forma más eficiente.

FOL

Todos los integrantes del grupo han colaborado en el proyecto desarrollando distintas habilidades sociales. Hemos mantenido una comunicación fluida para intercambiar ideas, hemos respetado y valorado las opiniones de los demás, y hemos trabajado de manera conjunta para lograr un buen resultado.

Además, hemos sabido resolver los pequeños desacuerdos mediante el diálogo y los acuerdos, y cada uno ha asumido su parte de responsabilidad, contribuyendo a la organización y al buen desarrollo del trabajo.

Automatización y Reglas de Negocio

1. Creación del procedimiento

```
SET GLOBAL event_scheduler = ON;
```

```
DELIMITER $$  
CREATE PROCEDURE InsertarTicketManual (  
    IN p_correo VARCHAR(50),  
    IN p_nombre VARCHAR(50),  
    IN p_apellido1 VARCHAR(50),  
    IN p_apellido2 VARCHAR(50),  
    IN p_telefono VARCHAR(20),  
    IN p_titulo VARCHAR(200),  
    IN p_descripcion VARCHAR(200),  
    IN p_id_empleado INT  
)  
MODIFIES SQL DATA  
BEGIN  
    DECLARE v_id_cliente INT DEFAULT NULL;  
    DECLARE v_contador INT DEFAULT 0;  
  
    SELECT COUNT(*) INTO v_contador  
    FROM clientes  
    WHERE correo = p_correo;  
  
    IF v_contador = 0 THEN  
        INSERT INTO clientes (nombre, apellido1, apellido2, correo, telefono)  
        VALUES (p_nombre, p_apellido1, p_apellido2, p_correo, p_telefono);  
        SET v_id_cliente = LAST_INSERT_ID();  
    ELSE  
        SELECT id_cliente INTO v_id_cliente  
        FROM clientes  
        WHERE correo = p_correo;  
    END IF;  
END IF;
```

```
INSERT INTO ticket (id_cliente_fk2, id_empleado_fk2, titulo, descripcion, prioridad, categoria, estados, fecha_hora_inicial)  
VALUES (v_id_cliente, p_id_empleado, p_titulo, p_descripcion, 'Media', 'Hardware', 'Abierto', NOW());  
SET @ultimo_ticket = LAST_INSERT_ID();  
  
END $$  
DELIMITER ;
```

Activamos el programador de eventos para que MySQL pueda ejecutar tareas automáticas.

Creamos un procedimiento, que se puede reutilizar más adelante, para crear un ticket nuevo para un cliente, en caso de que exista. Le pasamos todos los datos de entrada necesarios para que ejecute el procedimiento. Además, indicamos que el procedimiento modifica datos en la base de datos mediante operaciones como INSERT.

Añadimos una variable que va a almacenar cuántos clientes existen en el correo proporcionado.

Se consulta la tabla Clientes para verificar si ya existe un cliente con el correo indicado y se guarda en "v_contador_correo". Si la variable del contador es 0, significa que el cliente no existe y se registra el nuevo cliente a la tabla Clientes.

Después de insertar o localizar el cliente, se obtiene su ID y se almacena en la variable "v_id_cliente".

Por último, se crea un nuevo ticket y se guarda el ID del ticket en la variable de sesión de @ultimo_ticket.

2. Creación del trigger

```
DELIMITER $$
CREATE TRIGGER estado_ticket
  BEFORE UPDATE ON Ticket
  FOR EACH ROW
BEGIN
  IF NEW.estados = 'Cerrado' THEN
    IF NOT EXISTS (
      SELECT 1
      FROM mensaje
      WHERE id_ticket_fk1 = NEW.id_ticket
    ) THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'No se puede cerrar un ticket sin historial de mensajes';
    END IF;
  END IF;
END $$
```

Con este TRIGGER validamos que un ticket no pueda cerrarse si no tiene mensajes registrados en su historial.

Si el nuevo estado del ticket es "Cerrado", comprueba si existen mensajes asociados a ese ticket. Si no tiene mensajes, se genera un error y te impide seguir con el procedimiento.

3. Creación de tabla

```
CREATE TABLE IF NOT EXISTS Auditoria_Tickets (  
    id_auditoria INT NOT NULL AUTO_INCREMENT,  
    id_ticket INT NOT NULL,  
    estado_antiguo ENUM('Abierto','En proceso','Cerrado','Archivado') NOT NULL,  
    estado_nuevo ENUM('Abierto','En proceso','Cerrado','Archivado') NOT NULL,  
    fecha_cambio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (id_auditoria),  
    CONSTRAINT FK_ticket_auditoria  
        FOREIGN KEY (id_ticket)  
        REFERENCES Ticket(id_ticket)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
)  
  
ENGINE = InnoDB;
```

Creamos una nueva tabla para guardar los cambios realizados en el estado de los tickets. Guardamos tanto el estado antiguo como el estado nuevo y registramos la fecha y hora exacta de cuándo se efectuó el cambio.

Establecemos una relación entre la tabla “Auditoria Ticket” y la tabla Tickets.

```
DELIMITER $$  
CREATE TRIGGER auditoria_cambio_estado  
    AFTER UPDATE ON Ticket  
    FOR EACH ROW  
BEGIN  
    IF OLD.estados <> NEW.estados THEN  
        INSERT INTO Auditoria_Tickets (id_ticket, estado_antiguo, estado_nuevo, fecha_cambio)  
        VALUES (OLD.id_ticket, OLD.estados, NEW.estados, NOW());  
    END IF;  
END $$
```

Creamos un TRIGGER que registre automáticamente cualquier cambio de estado en los tickets dentro de la tabla Auditoria_Tickets.

Se ejecuta después de actualizar la tabla Tickets y sólo actúa si el ticket realmente cambió e inserta un registro en Auditoria_Tickets con la información del cambio.

4. Creación de evento

```
DELIMITER $$
CREATE EVENT archivar_ticket
  ON SCHEDULE EVERY 1 DAY
  STARTS TIMESTAMP(CURRENT_DATE + INTERVAL 1 DAY, '03:00:00')
  DO
BEGIN
  UPDATE Ticket
  SET estados = 'Archivado'
  WHERE estados = 'Cerrado'
    AND fecha_hora_final <= NOW() - INTERVAL 1095 DAY;
END $$
```

Por último, un evento que automatiza el archivado de tickets cerrados que llevan mucho tiempo sin actividad.

El evento se ejecuta una vez al día y empieza mañana a las 3 a.m, para esto le pusimos la fecha actual, le sumamos un día para que empezara mañana a las 00:00 y la cambiamos a las 3:00 a.m.

Cambia el estado a “Archivado” cuando el estado sea “Cerrado” desde hace más de 3 años.

Batería de Pruebas

Insertar Ticket

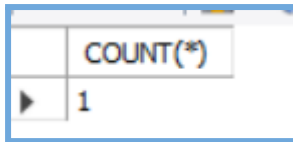
```
CALL InsertarTicketManual(  
    'cliente.nuevo@email.com',  
    'Juan',  
    'García',  
    'López',  
    '600123456',  
    'Mi PC no arranca',  
    'El ordenador no enciende desde esta mañana',  
    1  
);
```

id_ticket	titulo	descripcion	fecha_hora_inicial	categoria	prioridad	estados	fecha_hora_final	id_cliente_fk2	id_empleado_fk2
2	Mi PC no arranca	El ordenador no enciende desde esta mañana	2026-05-25 19:07:02	Hardware	Media	Abierto	0000	1	1

Al llamar al procedimiento con los datos del cliente nuevo, podemos ver que se ejecuta correctamente, insertando el cliente en la tabla clientes y creando un nuevo ticket asociado a él.

Insertar Ticket con ID existente

```
CALL InsertarTicketManual(  
    'cliente.nuevo@email.com',  
    'Juan',  
    'García',  
    'López',  
    '600123456',  
    'El ratón no funciona',  
    'El ratón dejó de responder',  
    1  
);  
  
SELECT COUNT(*) FROM clientes WHERE correo = 'cliente.nuevo@email.com';
```

id_ticket	titulo	descripcion	fecha_hora_inicial	categoria	prioridad	estados	fecha_hora_final	id_cliente_fk2	id_empleado_fk2
1	Mi PC no arranca	El ordenador no enciende desde esta mañana	2026-05-25 19:05:50	Hardware	Media	Abierto	NULL	1	1
2	Mi PC no arranca	El ordenador no enciende desde esta mañana	2026-05-25 19:07:02	Hardware	Media	Abierto	NULL	1	1
3	El ratón no funciona	El ratón dejó de responder	2026-05-25 19:55:24	Hardware	Media	Abierto	NULL	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Se ejecuta el procedimiento con el mismo correo de la prueba anterior. El resultado muestra que solo existe un registro en la tabla clientes con ese correo, lo que demuestra que el procedimiento detectó que el cliente ya existía y reutilizó su ID en lugar de crear un duplicado. Aun así, se creó un nuevo ticket asociado a ese cliente.

Intentar cerrar un ticket sin historial de mensaje

```
SET @id_prueba = (SELECT MAX(id_ticket) FROM ticket);
UPDATE ticket SET estados = 'Cerrado' WHERE id_ticket = @id_prueba;
```

63 20:05:57 UPDATE ticket SET estados = 'Cerrado' WHERE id_ticket = @id_prueba Error Code: 1644. No se puede cerrar un ticket sin historial de mensajes

Al intentar cerrar el ticket sin un historial de mensajes nos salta un error porque el trigger estado_ticket se activa justo antes de que se ejecute el UPDATE. Comprueba si ese ticket tiene algún mensaje en la tabla mensaje y, si no tiene ninguno, devuelve el error.

Registro de cambios de estados del ticket

```
INSERT INTO mensaje (id_mensaje, texto, fecha_hora_enviado, id_empleado_fk1, id_ticket_fk1)
VALUES (1, 'Problema resuelto', NOW(), 1, @id_prueba);

UPDATE ticket SET estados = 'Abierto' WHERE id_ticket = @id_prueba;
UPDATE ticket SET estados = 'Cerrado' WHERE id_ticket = @id_prueba;

SELECT * FROM Auditoria_Tickets;
```

	id_auditoria	id_ticket	estado_antiguo	estado_nuevo	fecha_cambio
	1	5	Cerrado	Abierto	2026-05-25 20:15:42
	2	5	Abierto	Cerrado	2026-05-25 20:15:46
	NULL	NULL	NULL	NULL	

Se cambia el estado del ticket de Cerrado a Abierto y luego de Abierto a Cerrado. La tabla Auditoria_Tickets registra automáticamente ambos cambios, guardando el ID del ticket, el estado anterior, el estado nuevo y la fecha y hora exacta del cambio.

Evento

Db	Name	Definer	Time zone	Type	Execute at	Interval value	Interval field	Starts	Ends	Status	Originator	character_set_client	collation_connection
hermes_it	archivar_ticket	root@localhost	SYSTEM	RECURRING	NULL	1	DAY	2026-05-26 03:00:00	NULL	ENABLED	1	utf8mb4	utf8mb4_0900_ai

Podemos comprobar que el evento archivar_ticket se ha creado correctamente y está en el sistema. Este evento se ejecutará automáticamente todos los días a las 3:00 y cambiará el estado de los tickets a 'Archivado' si llevan más de 3 años cerrados, sin necesidad de realizarlo manualmente.

Motor de Extracción y Transformación a XML

1. Extractor

```

1 import mysql.connector
2 import xml.etree.ElementTree as ET
3 from datetime import datetime
4 from funciones_sla_enfado import calcular_alerta_sla, detectar_enfado
5
6 log = open("extractor_log.txt", "a", encoding="utf-8")
7 contador = 0
8 fallidos = 0
9
10 cursor = None
11 connection = None
12 try:
13     connection = mysql.connector.connect(
14         host="localhost",
15         user="hermesadamin",
16         password="1234",
17         database="Hermes_IT"
18     )
19
20     cursor = connection.cursor()
21
22     cursor.execute("""SELECT T.id_ticket, T.titulo, T.descripcion, T.fecha_hora_inicial, T.categoria, T.fecha_hora_final, T.id_cliente_fk2, T.id_empleado_fk2, C.id_cliente, C.nombre, C.apellido1,
23 FROM ticket T JOIN clientes C
24 ON T.id_cliente_fk2 = C.id_cliente
25 JOIN operadores O
26 ON T.id_empleado_fk2 = O.id_empleado
27 WHERE T.estado = 'Archivado'""")
28
29     registro = cursor.fetchall()
30
31 except mysql.connector.Error:
32     print("Error no se puede conectar a la BD")
33     log.write(f"[{datetime.now()}] ERROR de conexión\n")
34 else:
35     for ticket in registro:
36         T_id_ticket, T_titulo, T_descripcion, T_fecha_hora_inicial, T_categoria, T_fecha_hora_final, T_id_cliente_fk2, T_id_empleado_fk2, C_id_cliente, C_nombre, C_apellido1, C_apellido2, C_correo,
37         try:

```

Empezamos abriendo un fichero del tipo que añada cosas al final y no sobrescriba la información y declaramos dos contadores: uno para los tickets exportados correctamente y otro para los fallidos.

Antes del try, ponemos una variable cursor y connection a 'None' por si acaso la conexión falla, las variables siguen existiendo. Luego, conectamos la base de datos en localhost.

Hicimos un JOIN entre Ticket, Clientes y Operadores para obtener toda la información de tickets cuyo estado es 'Archivado'. Con el comando 'fetchall()' guardamos todos los resultados.

```
registro = cursor.fetchall()

except mysql.connector.Error:
    print("Error no se puede conectar a la BD")
    log.write(f"[{datetime.now()}] ERROR de conexión\n")
else:
    for ticket in registro:
        T_id_ticket, T_titulo, T_descripcion, T_fecha_hora_inicial, T_categoria,
        T_fecha_hora_final, T_id_cliente_fk2, T_id_empleado_fk2, C_id_cliente, C_nombre, C_apellido1,
        C_apellido2, C_correo, C_telefono, O_id_empleado, O_nombre, O_correo, O_id_departamento_fk1 = ticket
        try:
```

Si la conexión falla, manda un mensaje de error, sino la conexión fue exitosa. En este último caso, se crean tuplas de 18 campos con todos los datos de estas tablas.

```
else:
    for ticket in registro:
        T_id_ticket, T_titulo, T_descripcion, T_fecha_hora_inicial, T_categoria, T_fecha_hora_final, T_id_cliente_fk2, T_id_empleado_fk2, C_id_cliente_fk1 = ticket
        try:
            cursor.execute(f"""
            SELECT texto, fecha_hora_enviado, id_cliente_fk1, id_empleado_fk1
            FROM Mensaje
            WHERE id_ticket_fk1 = {T_id_ticket}
            ORDER BY fecha_hora_enviado ASC
            """)
            mensajes = cursor.fetchall()

            raiz = ET.Element("ticket")
            raiz.set("id", str(T_id_ticket))

            textos_cliente = []
            for tupla in mensajes:
                texto_mensaje, fecha_mensaje, id_cli_mensaje, id_emp_mensaje = tupla
                if id_cli_mensaje is not None:
                    textos_cliente.append(texto_mensaje)

            if detectar_enfado(textos_cliente):
                raiz.set("cliente_enfadado", "si")
```

En la siguiente consulta, por cada ticket, va a la tabla Mensaje y trae todos los mensajes asociados ordenados de forma cronológica.

La raíz de nuestro XML se va a llamar 'ticket' y su atributo va a ser el ID del ticket.

Recorremos los mensajes para extraer solo los del cliente, es decir, los que tienen el ID del cliente no NULL y se lo pasamos a la función 'detectar_enfado'. Si devuelve True, añade el atributo 'cliente_enfadado = si'.

```

etiqueta_titulo = ET.Element("titulo")
etiqueta_titulo.text = T_titulo
raiz.append(etiqueta_titulo)

etiqueta_descripcion = ET.Element("descripcion")
etiqueta_descripcion.text = T_descripcion
raiz.append(etiqueta_descripcion)

etiqueta_fecha_hora_inicial = ET.Element("fecha_hora_inicial")
etiqueta_fecha_hora_inicial.text = str(T_fecha_hora_inicial)
raiz.append(etiqueta_fecha_hora_inicial)

etiqueta_categoria = ET.Element("categoria")
etiqueta_categoria.text = T_categoria
raiz.append(etiqueta_categoria)

etiqueta_fecha_hora_final = ET.Element("fecha_hora_final")
etiqueta_fecha_hora_final.text = str(T_fecha_hora_final)
raiz.append(etiqueta_fecha_hora_final)

etiqueta_id_cliente_fk2 = ET.Element("id_cliente_fk2")
etiqueta_id_cliente_fk2.text = str(T_id_cliente_fk2)
raiz.append(etiqueta_id_cliente_fk2)

etiqueta_id_empleado_fk2 = ET.Element("id_empleado_fk2")
etiqueta_id_empleado_fk2.text = str(T_id_empleado_fk2)
raiz.append(etiqueta_id_empleado_fk2)

```

Este patrón se repite para todos los campos: se crea el elementar y se pone en la raíz.

```

dias_retraso = calcular_alerta_sla(T_fecha_hora_inicial, T_fecha_hora_final)
if dias_retraso is not None:
    etiqueta_sla = ET.Element("alerta_sla")
    etiqueta_sla.set("dias_retraso", str(dias_retraso))
    etiqueta_sla.text = f"Cerrado con {dias_retraso} días de retraso"
    raiz.append(etiqueta_sla)

etiqueta_cliente = ET.Element("cliente")

etiqueta_nombre_cli = ET.Element("nombre")
etiqueta_nombre_cli.text = C_nombre
etiqueta_cliente.append(etiqueta_nombre_cli)

etiqueta_apellidos_cli = ET.Element("apellido1")
etiqueta_apellidos_cli.text = C_apellido1
etiqueta_cliente.append(etiqueta_apellidos_cli)

etiqueta_apellidos_cli2 = ET.Element("apellido2")
etiqueta_apellidos_cli2.text = C_apellido2
etiqueta_cliente.append(etiqueta_apellidos_cli2)

etiqueta_correo_cli = ET.Element("correo")
etiqueta_correo_cli.text = C_correo
etiqueta_cliente.append(etiqueta_correo_cli)

etiqueta_telefono_cli = ET.Element("telefono")
etiqueta_telefono_cli.text = str(C_telefono)
etiqueta_cliente.append(etiqueta_telefono_cli)

```

Restamos la hora inicial y la hora final, si suma más de 7 se crea una etiqueta llamada 'alerta_sla' con el atributo que dice los días de retraso y un texto descriptivo, sino no añade nada la XML.

Creamos un elemento 'cliente' y le añadimos sub-etiquetas con sus datos.

```

raiz.append(etiqueta_cliente)

etiqueta_operador = ET.Element("operador")
if O_nombre is not None:
    etiqueta_nombre_op = ET.Element("nombre")
    etiqueta_nombre_op.text = O_nombre
    etiqueta_operador.append(etiqueta_nombre_op)
    etiqueta_correo_op = ET.Element("correo")
    etiqueta_correo_op.text = O_correo
    etiqueta_operador.append(etiqueta_correo_op)
raiz.append(etiqueta_operador)

etiqueta_historial = ET.Element("historial")
for tupla in mensajes:
    texto_mensaje, fecha_mensaje, id_cli_mensaje, id_emp_mensaje = tupla
    etiqueta_mensaje = ET.Element("mensaje")
    if id_cli_mensaje is not None:
        autor = "cliente"
    else:
        autor = "operador"
    etiqueta_mensaje.set("autor", autor)
    etiqueta_mensaje.set("fecha", str(fecha_mensaje))
    etiqueta_mensaje.text = texto_mensaje
    etiqueta_historial.append(etiqueta_mensaje)
raiz.append(etiqueta_historial)

```

Creamos la etiqueta 'historial' que contiene un mensaje por cada mensaje del ticket. Cada mensaje tiene dos atributos: autor (cliente u operador) y fecha.

```

ET.indent(raiz, space="  ")
xml_texto = ET.tostring(raiz, encoding="unicode")
xml_texto = '<?xml version="1.0" encoding="UTF-8"?>\n' + xml_texto

ruta = f"ticket_{T_id_ticket}.xml"
with open(ruta, "w", encoding="utf-8") as archivo:
    archivo.write(xml_texto)

contador += 1

except:
    fallidos += 1
    log.write(f"[{datetime.now()}] ERROR con el ticket {T_id_ticket}\n")

```

ET.indent (raiz, space=" ") se utiliza para dar formato y sangría al árbol XML, haciendo que se vea más ordenado y legible, como los XML que normalmente vemos en clase.

Por otro lado, ET.tostring (raiz, encoding="unicode") convierte el árbol XML en una cadena de texto (str). Al usar el formato Unicode, se pueden representar correctamente las tildes y

los acentos, convirtiéndolos en texto normal. Esto nos permite guardar, imprimir o enviar el XML como si fuera un texto.

Esta parte del XML tuvimos que investigarla, ya que en clase todavía no la habíamos visto.

```
if cursor is not None:
    cursor.close()
if connection is not None and connection.is_connected():
    connection.close()

log.write(f"[{datetime.now()}] XML generados: {contador} y fallidos: {fallidos}\n")
log.close()
```

Cerramos el cursor y la conexión solo si llegaron a crearse. Además, se escribe un resumen final y se cierra el log.

2. Funciones de enfado

```
def calcular_alerta_sla(fecha_inicial, fecha_final):
    """Devuelve los días de retaso si el ticket se cerró después de 7 días, o None """
    if fecha_inicial is None or fecha_final is None:
        return None
    dias = (fecha_final - fecha_inicial).days
    if dias > 7:
        return dias - 7
    return None
```

En esta función, si el ticket se cerró después de 7 días devuelve los días que se ha retrasado el cierre, sino retorna 'None'.

```
def detectar_enfado(mensajes_cliente):
    """Devuelve True si algún mensaje del cliente contiene palabras de enfado."""
    palabras = ["incompetentes", "denuncia", "vergüenza", "lento"]
    texto_junto = " ".join(mensajes_cliente).lower()
    for palabra in palabras:
        if palabra in texto_junto:
            return True
    return False
```

Hicimos una función que devuelva 'True' si algún mensaje del cliente contiene palabras de enfado. Si termina el bucle sin encontrar ninguna, devuelve 'False'.

3. Modificar mensajes del ticket

```
import mysql.connector
import random

palabras = ["incompetentes", "denuncia", "vergüenza", "lento"]
connection = mysql.connector.connect(
    host="localhost",
    user="hermesadamin",
    password="1234",
    database="Hermes_IT"
)
cursor = connection.cursor()

cursor.execute("SELECT id_ticket, descripcion FROM ticket")
tickets = cursor.fetchall()
```

Nos conectamos a la base de datos de manera local y traemos el ID y la descripción de todos los tickets y lo guardamos en memoria.

```
for id_ticket, descripcion in tickets:
    numero_aleatorio = random.random()
    if numero_aleatorio < 0.10:
        palabra_random = random.choice(palabras)
        if descripcion is None:
            descripcion = ""
        nueva_descripcion = descripcion + " " + palabra_random
        cursor.execute(
            f"UPDATE mensaje SET texto = '{nueva_descripcion}' WHERE id_mensaje = {id_ticket}"
        )
    connection.commit()
```

Por cada ticket, generamos un número aleatorio entre el 0.0 y el 1.0. Si el número es menor que 0.10 se genera una palabra aleatoria y se añade a la descripción. Por último, se actualiza la tabla Ticket con la nueva descripción.

Es decir, hay un 10% de probabilidades de que se genere esta palabra.

Confirmamos todos los cambios en la base de datos.

```
for id_ticket, fecha_inicial in tickets:
    if fecha_inicial is None:
        continue

    inicio = fecha_inicial + timedelta(days=1)
    fin = fecha_inicial + timedelta(days=20)

    nueva_fecha_final = fake.date_time_between_dates(
        datetime_start=inicio,
        datetime_end=fin
    )

    cursor.execute(
        f"UPDATE ticket SET fecha_hora_final = '{nueva_fecha_final}' WHERE id_ticket = {id_ticket}"
    )

    connection.commit()
    cursor.close()
    connection.close()

except:
    print("Error con la base de datos")
```

Hacemos un bucle donde se recorren todos los tickets archivados y se les asigna una fecha y hora final de manera aleatoria entre 1 y 20 días después de la fecha inicial.

Por último, lo guardamos todo y cerramos el cursor y la conexión.

4. Tickets generados

```
ticket_107785.xml > ticket
<?xml version="1.0" encoding="UTF-8"?>
<ticket id="107785" cliente_enfadado="si">
  <titulo>Base dólares su violencia blanco.</titulo>
  <descripcion>Titulo corazón opinión. Todo lucha personas lleva presente.</descripcion>
  <fecha_hora_inicial>2026-04-11 09:52:06</fecha_hora_inicial>
  <categoria>Redes</categoria>
  <fecha_hora_final>2026-04-28 18:55:25</fecha_hora_final>
  <id_cliente_fk2>908011</id_cliente_fk2>
  <id_empleado_fk2>712268</id_empleado_fk2>
  <alerta_sla_dias_retraso="10">Cerrado con 10 días de retraso</alerta_sla>
  <cliente>
    <nombre>Visitación</nombre>
    <apellido1>Becerra</apellido1>
    <apellido2>Barranco</apellido2>
    <correo>lorenzoaroca@example.net</correo>
    <telefono>+34844 164 518</telefono>
  </cliente>
  <operador>
    <nombre>Paola</nombre>
    <correo>aviladelfina@example.net</correo>
  </operador>
  <historial>
    <mensaje autor="cliente" fecha="2026-05-08 08:12:40">Demasiado iglesia habla historia dan. Partir recuerdo escuela local congreso lugar.</mensaje>
    <mensaje autor="cliente" fecha="2026-05-22 06:32:20">Objeto esas calle debe hijos. Han ocho entrada vía objeto. incompetentes</mensaje>
  </historial>
</ticket>
```

```

<?xml version="1.0" encoding="UTF-8"?>
<ticket id="118640">
  <titulo>Permite niños.</titulo>
  <descripcion>Consumo capacidad oficial proyectos. Lado todas buenos ese.</descripcion>
  <fecha_hora_inicial>2025-06-23 04:21:29</fecha_hora_inicial>
  <categoria>Software</categoria>
  <fecha_hora_final>2025-07-05 00:35:10</fecha_hora_final>
  <id_cliente_fk2>673326</id_cliente_fk2>
  <id_empleado_fk2>395036</id_empleado_fk2>
  <alerta_sla_dias_retraso="4">Cerrado con 4 días de retraso</alerta_sla>
  <cliente>
    <nombre>Brunilda</nombre>
    <apellido1>Arenas</apellido1>
    <apellido2>Juárez</apellido2>
    <correo>acamps@example.org</correo>
    <telefono>+34868701739</telefono>
  </cliente>
  <operador>
    <nombre>Santos</nombre>
    <correo>gutierrezandres@example.org</correo>
  </operador>
  <historial>
    <mensaje autor="cliente" fecha="2025-09-19 11:59:43">Era casa ex el. Más conocimiento medidas datos fuerte cine izquierda. Principio hasta humana baja.</mensaje>
  </historial>
</ticket>

```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <ticket id="254683" cliente_enfadado="si">
3   <titulo>Proyectos blanco afirmó.</titulo>
4   <descripcion>Has fondo capaz frente estructura final fuerte. Puerto medio se imágenes lejos ha.</descripcion>
5   <fecha_hora_inicial>2025-02-02 13:54:48</fecha_hora_inicial>
6   <categoria>Software</categoria>
7   <fecha_hora_final>2025-02-09 10:20:03</fecha_hora_final>
8   <id_cliente_fk2>674382</id_cliente_fk2>
9   <id_empleado_fk2>261277</id_empleado_fk2>
10  <cliente>
11    <nombre>Pia</nombre>
12    <apellido1>Ocaña</apellido1>
13    <apellido2>Baños</apellido2>
14    <correo>corrallina@example.com</correo>
15    <telefono>+34746 857 174</telefono>
16  </cliente>
17  <operador>
18    <nombre>Leandra</nombre>
19    <correo>clementinafrutos@example.org</correo>
20  </operador>
21  <historial>
22    <mensaje autor="cliente" fecha="2025-02-24 02:31:20">Temas aparece resultados distintos pese. Capacidad derechos esto cuadro dar. Don medios elementos.</mensaje>
23    <mensaje autor="operador" fecha="2025-04-21 17:17:23">Pequeña santiago derecho grupos. Ninguna hemos quería. América interés ir visita.</mensaje>
24    <mensaje autor="cliente" fecha="2025-05-04 21:22:09">Trabajar asimismo cual niño agua final falta. Información mayor comunidad niños obstante partes. vergüenza</mensaje>
25  </historial>
26 </ticket>

```